A long listing

The listing below is longer than a page. It splits between two lines and goes on in a box on the next page.

```
/// Part 1: reads the stock of one warehouse and reports what is below the line it was asked
to keep, which is a long comment on purpose, so that it wraps inside the box and takes a
second line.
fn stock_1(warehouse: &Warehouse, floor: u32) -> Vec<Item> {
    let mut low = Vec::new();
    for item in warehouse.items() {
        if item.count < floor {
            low.push(item.clone());
        }
    }

    // a tab here, as an editor with tabs would write the line
    low.sort_by_key(|item| item.count);
    low
}

/// Part 2: reads the stock of one warehouse and reports what is below the line it was asked
to keep, which is a long comment on purpose, so that it wraps inside the box and takes a
second line.
fn stock_2(warehouse: &Warehouse, floor: u32) -> Vec<Item> {
    let mut low = Vec::new();
    for item in warehouse.items() {
        if item.count < floor {
            low.push(item.clone());
        }
    }

    // a tab here, as an editor with tabs would write the line
    low.sort_by_key(|item| item.count);
    low
}

/// Part 3: reads the stock of one warehouse and reports what is below the line it was asked
to keep, which is a long comment on purpose, so that it wraps inside the box and takes a
second line.
fn stock_3(warehouse: &Warehouse, floor: u32) -> Vec<Item> {
    let mut low = Vec::new();
    for item in warehouse.items() {
        if item.count < floor {
            low.push(item.clone());
        }
    }

    // a tab here, as an editor with tabs would write the line
    low.sort_by_key(|item| item.count);
    low
}

/// Part 4: reads the stock of one warehouse and reports what is below the line it was asked
to keep, which is a long comment on purpose, so that it wraps inside the box and takes a
second line.
fn stock_4(warehouse: &Warehouse, floor: u32) -> Vec<Item> {
    let mut low = Vec::new();
    for item in warehouse.items() {
        if item.count < floor {
            low.push(item.clone());
        }
    }

    // a tab here, as an editor with tabs would write the line
    low.sort_by_key(|item| item.count);
    low
}
```

Lpdf

```
/// Part 5: reads the stock of one warehouse and reports what is below the line it was asked
to keep, which is a long comment on purpose, so that it wraps inside the box and takes a
second line.
fn stock_5(warehouse: &Warehouse, floor: u32) -> Vec<Item> {
    let mut low = Vec::new();
    for item in warehouse.items() {
        if item.count < floor {
            low.push(item.clone());
        }
    }

    // a tab here, as an editor with tabs would write the line
    low.sort_by_key(|item| item.count);
    low
}

/// Part 6: reads the stock of one warehouse and reports what is below the line it was asked
to keep, which is a long comment on purpose, so that it wraps inside the box and takes a
second line.
fn stock_6(warehouse: &Warehouse, floor: u32) -> Vec<Item> {
    let mut low = Vec::new();
    for item in warehouse.items() {
        if item.count < floor {
            low.push(item.clone());
        }
    }

    // a tab here, as an editor with tabs would write the line
    low.sort_by_key(|item| item.count);
    low
}

/// Part 7: reads the stock of one warehouse and reports what is below the line it was asked
to keep, which is a long comment on purpose, so that it wraps inside the box and takes a
second line.
fn stock_7(warehouse: &Warehouse, floor: u32) -> Vec<Item> {
    let mut low = Vec::new();
    for item in warehouse.items() {
        if item.count < floor {
            low.push(item.clone());
        }
    }

    // a tab here, as an editor with tabs would write the line
    low.sort_by_key(|item| item.count);
    low
}

/// Part 8: reads the stock of one warehouse and reports what is below the line it was asked
to keep, which is a long comment on purpose, so that it wraps inside the box and takes a
second line.
fn stock_8(warehouse: &Warehouse, floor: u32) -> Vec<Item> {
    let mut low = Vec::new();
    for item in warehouse.items() {
        if item.count < floor {
            low.push(item.clone());
        }
    }

    // a tab here, as an editor with tabs would write the line
    low.sort_by_key(|item| item.count);
    low
}

/// Part 9: reads the stock of one warehouse and reports what is below the line it was asked
to keep, which is a long comment on purpose, so that it wraps inside the box and takes a
second line.
fn stock_9(warehouse: &Warehouse, floor: u32) -> Vec<Item> {
```

Lpdf

```
    let mut low = Vec::new();
    for item in warehouse.items() {
        if item.count < floor {
            low.push(item.clone());
        }
    }

    // a tab here, as an editor with tabs would write the line
    low.sort_by_key(|item| item.count);
    low
}

/// Part 10: reads the stock of one warehouse and reports what is below the line it was
asked to keep, which is a long comment on purpose, so that it wraps inside the box and takes
a second line.
fn stock_10(warehouse: &Warehouse, floor: u32) -> Vec<Item> {
    let mut low = Vec::new();
    for item in warehouse.items() {
        if item.count < floor {
            low.push(item.clone());
        }
    }

    // a tab here, as an editor with tabs would write the line
    low.sort_by_key(|item| item.count);
    low
}

/// Part 11: reads the stock of one warehouse and reports what is below the line it was
asked to keep, which is a long comment on purpose, so that it wraps inside the box and takes
a second line.
fn stock_11(warehouse: &Warehouse, floor: u32) -> Vec<Item> {
    let mut low = Vec::new();
    for item in warehouse.items() {
        if item.count < floor {
            low.push(item.clone());
        }
    }

    // a tab here, as an editor with tabs would write the line
    low.sort_by_key(|item| item.count);
    low
}

/// Part 12: reads the stock of one warehouse and reports what is below the line it was
asked to keep, which is a long comment on purpose, so that it wraps inside the box and takes
a second line.
fn stock_12(warehouse: &Warehouse, floor: u32) -> Vec<Item> {
    let mut low = Vec::new();
    for item in warehouse.items() {
        if item.count < floor {
            low.push(item.clone());
        }
    }

    // a tab here, as an editor with tabs would write the line
    low.sort_by_key(|item| item.count);
    low
}

/// Part 13: reads the stock of one warehouse and reports what is below the line it was
asked to keep, which is a long comment on purpose, so that it wraps inside the box and takes
a second line.
fn stock_13(warehouse: &Warehouse, floor: u32) -> Vec<Item> {
    let mut low = Vec::new();
    for item in warehouse.items() {
        if item.count < floor {
            low.push(item.clone());
        }
```

Lpdf

```
    }

    // a tab here, as an editor with tabs would write the line
    low.sort_by_key(|item| item.count);
    low
}
```

The sentence after the listing starts under its last box.